

CENTRALIZED LOGGING AND ANALYTICS SYSTEM

Services: EC2, CloudWatch Logs, CloudWatch Agent, S3, Athena, IAM

Description: Collect logs from multiple EC2 instances using CloudWatch Agent, centralize logs in CloudWatch Logs, store log data in Amazon S3, and analyze logs using Amazon Athena. Implement IAM-based access control for secure log management and monitoring.

Skills: Log Management, Cloud Monitoring, CloudWatch Logs, Athena Querying, S3 Storage, IAM Security, Troubleshooting, Log Analytics.

Definition: The Centralized Logging and Analytics System is designed to collect, store, monitor, and analyze application logs generated by multiple EC2 instances. CloudWatch Agent is installed on EC2 instances to continuously send logs to CloudWatch Logs. The collected logs are archived in Amazon S3 and analyzed using Amazon Athena through SQL queries. IAM policies are used to secure access to logs and analytics resources.

Scope of the Project: The scope of this project is to implement a centralized logging solution that enables efficient log collection, storage, monitoring, and analysis across multiple AWS resources.

Methodology

1. CloudWatch Logs + S3 + Athena (Recommended Method)

- Uses AWS managed services for log collection and analytics.
- Centralizes logs from multiple EC2 instances.
- Provides serverless log querying through Athena.
- No dedicated analytics servers required.
- Easy to scale and maintain.
- Cost-effective and suitable for AWS Free Tier projects.

2. EC2 + Local Log Files + Custom Scripts (Alternative Method)

Logs are stored directly on EC2 instances.

- Custom scripts are required to collect and analyze logs.
- No centralized monitoring capability.
- Higher operational effort and maintenance.
- Difficult to scale across multiple servers.

AWS Services Used in the Project

Amazon EC2: (Elastic Compute Cloud)

Definition: Amazon EC2 provides resizable virtual servers in the cloud.

Purpose: Hosts applications and generates log files.

Role: Acts as the source of application and system logs.

Amazon CloudWatch Logs

Definition: Amazon CloudWatch Logs is a monitoring service that collects and stores log files from AWS resources and applications.

Purpose: Centralizes logs from multiple EC2 instances.

Role: Receives logs from CloudWatch Agent and stores them in Log Groups and Log Streams.

Amazon CloudWatch Agent

Definition: CloudWatch Agent is software installed on EC2 instances that collects logs and system metrics.

Purpose: Transfers log files from EC2 instances to CloudWatch Logs.

Role: Continuously monitors and forwards log data.

Amazon S3 (Simple Storage Service)

Definition: Amazon S3 is an object storage service designed for durability and scalability.

Purpose: Stores archived log data for long-term retention.

Role: Acts as the centralized storage location for exported log data.

Amazon Athena

Definition: Amazon Athena is a serverless query service that enables SQL analysis of data stored in Amazon S3.

Purpose: Performs log analysis and reporting.

Role: Executes SQL queries against log files stored in S3.

AWS Identity and Access Management (IAM)

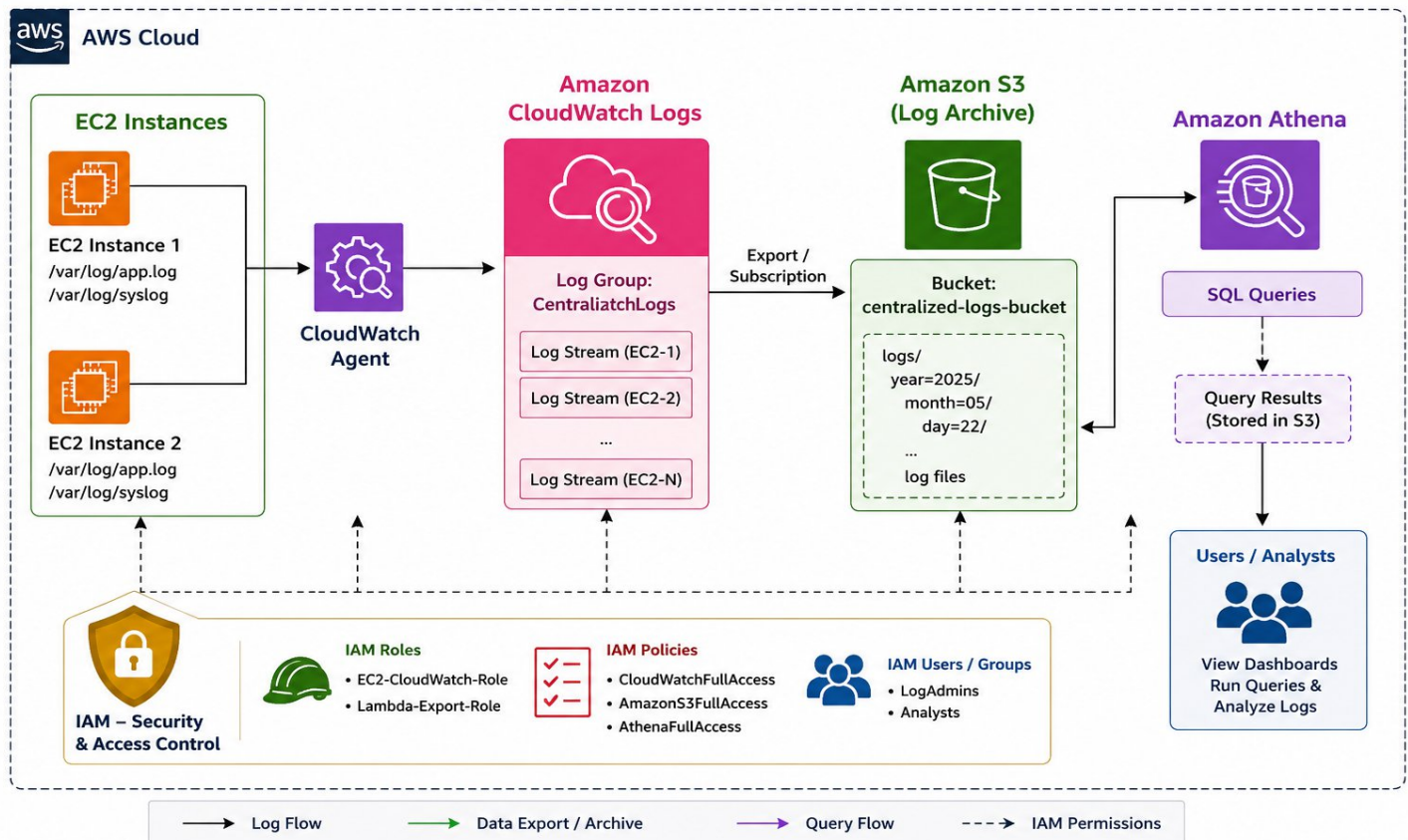
Definition: IAM enables secure access control for AWS resources.

Purpose: Controls permissions for accessing logs and analytics resources.

Role: Provides authentication and authorization for administrators and users.

Architecture Diagram

Centralized Logging and Analytics System Architecture



Implementation Steps

Step 1: Create IAM Role for EC2 and attach CloudWatchAgentServerPolicy.

- Open IAM Console.
- Create Role.
- Trusted Entity Type: AWS Service.
- Use Case: EC2.
- Attach Policy:

CloudWatchAgentServerPolicy

AmazonSSMManagedInstanceCore

- Role Name:

EC2-CloudWatch-Role

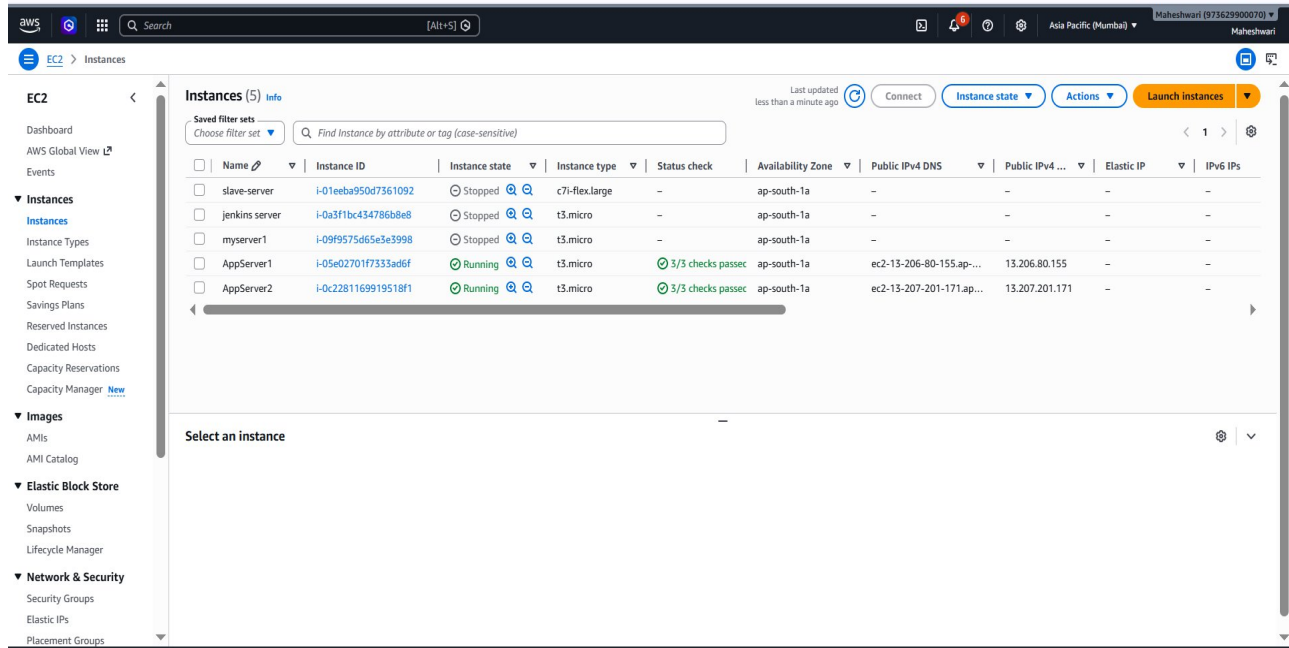
- Create Role.

The screenshot displays the AWS IAM console interface. The top navigation bar includes the AWS logo, a search bar, and user information for Maheshwari (973629900070). The left sidebar shows the 'Identity and Access Management (IAM)' menu with options like Dashboard, Access Management, Roles, Policies, IAM users, IAM user groups, Identity providers, Account settings, Root access management, and Temporary delegation requests. The main content area is titled 'EC2-CloudWatch-Role' and includes a description: 'Allows EC2 instances to call AWS services on your behalf.' Below this is a 'Summary' section with fields for Creation date (June 05, 2026, 17:16 (UTC+05:30)), ARN (arn:aws:iam::973629900070:role/EC2-CloudWatch-Role), Instance profile ARN (arn:aws:iam::973629900070:instance-profile/EC2-CloudWatch-Role), Last activity (7 minutes ago), and Maximum session duration (1 hour). There are 'Delete' and 'Edit' buttons in the top right. Below the summary is a tabbed interface with 'Permissions' selected, showing 'Permissions policies (2)' and a 'Simulate' button. Other tabs include 'Trust relationships', 'Tags', 'Last Accessed', and 'Revoke sessions'.

Step 2: Launch two Amazon Linux EC2 instances and attach the IAM role.

Open EC2 Console.

- Launch two Amazon Linux 2023 instances.
- Instance Type: t2. micro.
- Attach IAM Role: EC2-CloudWatch-Role.
- Launch Instances.



Step 3: Install CloudWatch Agent on both instances.

- Connect to each EC2 instance.
- Update packages.
- Install CloudWatch Agent.
- Verify successful installation.

```
all:
    all existing configs. Only apply to remove-config action.

-a: optionally restart after configuring the agent configuration
    this parameter is used for 'fetch-config', 'append-config', 'remove-config' action only.

-d: enable dual-stack endpoints for AWS API calls during config download
    this parameter is used for 'fetch-config', 'append-config' actions only.

-l: log level to set the agent to INFO, DEBUG, WARN, ERROR, or OFF
    this parameter is used for 'set-log-level' only.

[root@ip-172-31-8-218 ec2-user]# rpm -qa | grep amazon-cloudwatch-agent
amazon-cloudwatch-agent-1.300066.2-2.amzn2023.x86_64
[root@ip-172-31-8-218 ec2-user]# while true
do
echo "{\"server\":\"AppServer1\",\"level\":\"INFO\",\"message\":\"Application Running\",\"time\":\"$(date)\"}" >> /logs/app.log
sleep 10
done
```

i-05e02701f7333ad6f (AppServer1)
PublicIPs: 13.206.80.155 PrivateIPs: 172.31.8.218

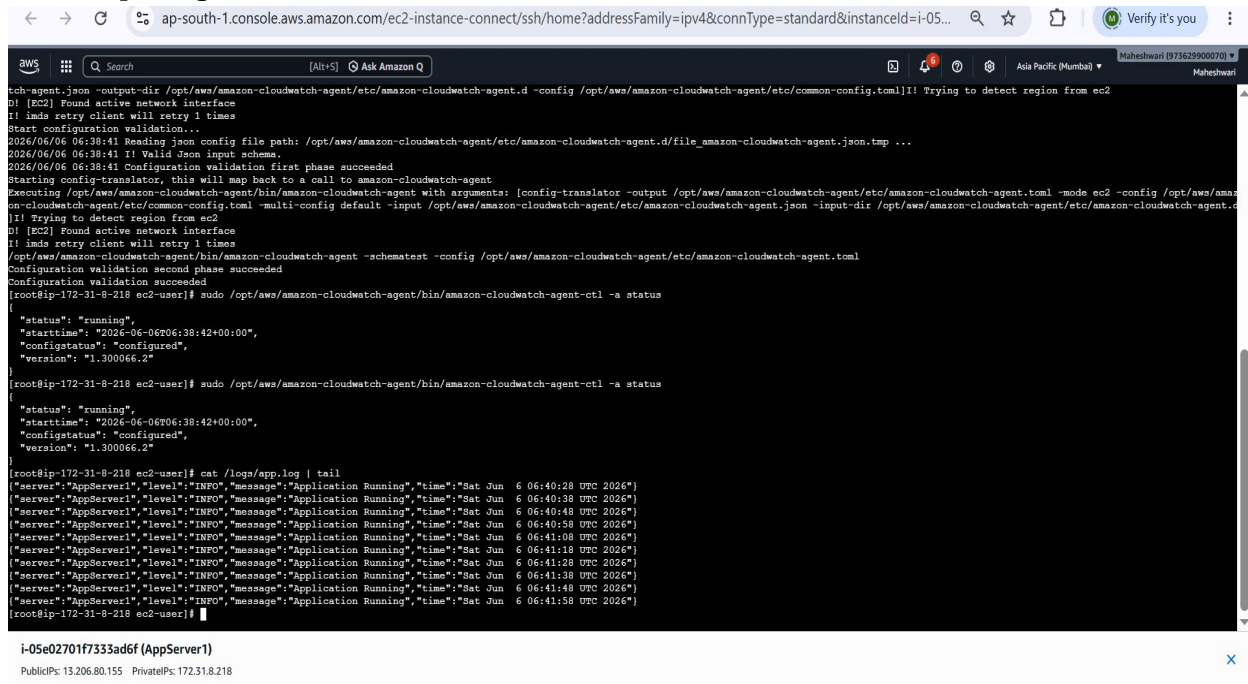
```
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

Last login: Sat Jun  6 06:20:53 2026 from 13.233.177.5
[ec2-user@ip-172-31-11-116 ~]$ sudo su
[root@ip-172-31-11-116 ec2-user]# rpm -qa | grep amazon-cloudwatch-agent
amazon-cloudwatch-agent-1.300066.2-2.amzn2023.x86_64
[root@ip-172-31-11-116 ec2-user]#
```

i-0c2281169919518f1 (AppServer2)
PublicIPs: 13.207.201.171 PrivateIPs: 172.31.11.116

Step 4: Create application log files and generate sample logs.

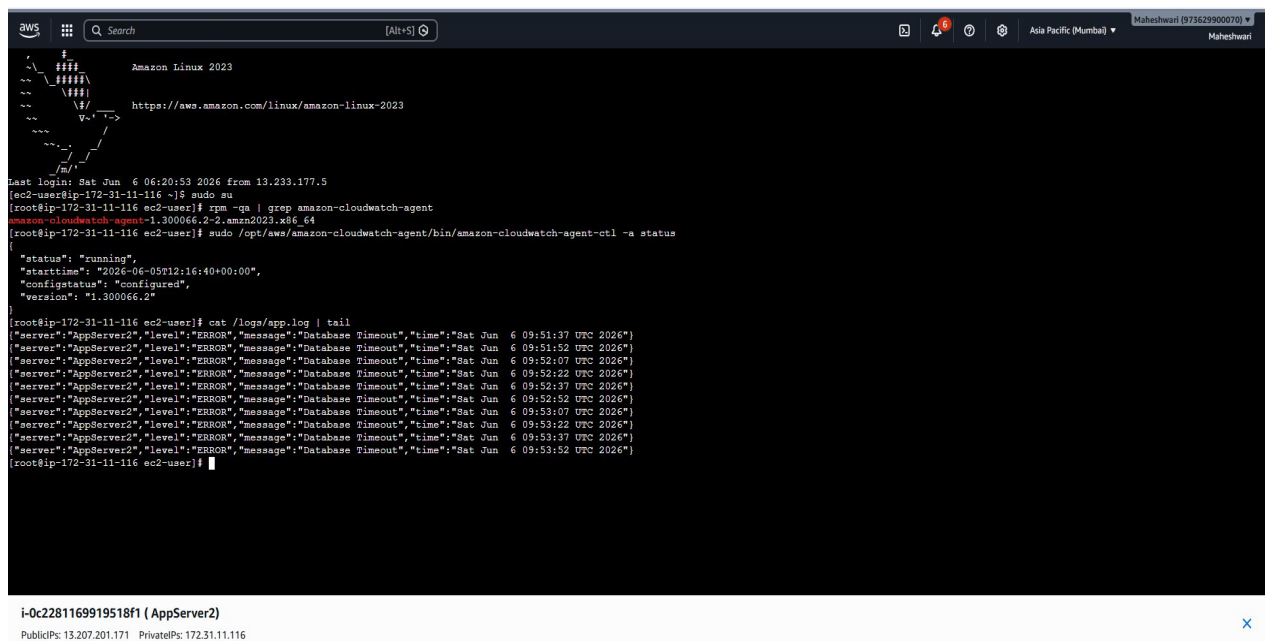
- Create log directory.
- Create application log file.
- Assign permissions.
- Generate sample log entries.



The screenshot shows an AWS console terminal window for an EC2 instance. The terminal output shows the installation of the Amazon CloudWatch agent. It starts with a command to install the agent, followed by a command to start it. The output shows the agent is running and its status is 'running'. The terminal also shows the contents of the log file, which contains several log entries for the application running.

```
root@ip-172-31-8-218 ec2-user:~# sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a status
{
  "status": "running",
  "starttime": "2026-06-06T06:38:42+00:00",
  "configstatus": "configured",
  "version": "1.300066.2"
}
root@ip-172-31-8-218 ec2-user:~# sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a status
{
  "status": "running",
  "starttime": "2026-06-06T06:38:42+00:00",
  "configstatus": "configured",
  "version": "1.300066.2"
}
root@ip-172-31-8-218 ec2-user:~# cat /logs/app.log | tail
{"server": "AppServer1", "level": "INFO", "message": "Application Running", "time": "Sat Jun 6 06:40:28 UTC 2026"}
{"server": "AppServer1", "level": "INFO", "message": "Application Running", "time": "Sat Jun 6 06:40:38 UTC 2026"}
{"server": "AppServer1", "level": "INFO", "message": "Application Running", "time": "Sat Jun 6 06:40:48 UTC 2026"}
{"server": "AppServer1", "level": "INFO", "message": "Application Running", "time": "Sat Jun 6 06:40:58 UTC 2026"}
{"server": "AppServer1", "level": "INFO", "message": "Application Running", "time": "Sat Jun 6 06:41:08 UTC 2026"}
{"server": "AppServer1", "level": "INFO", "message": "Application Running", "time": "Sat Jun 6 06:41:18 UTC 2026"}
{"server": "AppServer1", "level": "INFO", "message": "Application Running", "time": "Sat Jun 6 06:41:28 UTC 2026"}
{"server": "AppServer1", "level": "INFO", "message": "Application Running", "time": "Sat Jun 6 06:41:38 UTC 2026"}
{"server": "AppServer1", "level": "INFO", "message": "Application Running", "time": "Sat Jun 6 06:41:48 UTC 2026"}
{"server": "AppServer1", "level": "INFO", "message": "Application Running", "time": "Sat Jun 6 06:41:58 UTC 2026"}
root@ip-172-31-8-218 ec2-user:~#
```

i-05e02701f7333ad6f (AppServer1)
PublicIPs: 13.206.80.155 PrivateIPs: 172.31.8.218



The screenshot shows an AWS console terminal window for an EC2 instance. The terminal output shows the installation of the Amazon CloudWatch agent. It starts with a command to install the agent, followed by a command to start it. The output shows the agent is running and its status is 'running'. The terminal also shows the contents of the log file, which contains several log entries for the application running.

```
root@ip-172-31-11-116 ec2-user:~# sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a status
{
  "status": "running",
  "starttime": "2026-06-06T09:12:16+00:00",
  "configstatus": "configured",
  "version": "1.300066.2"
}
root@ip-172-31-11-116 ec2-user:~# cat /logs/app.log | tail
{"server": "AppServer2", "level": "ERROR", "message": "Database Timeout", "time": "Sat Jun 6 09:51:37 UTC 2026"}
{"server": "AppServer2", "level": "ERROR", "message": "Database Timeout", "time": "Sat Jun 6 09:51:52 UTC 2026"}
{"server": "AppServer2", "level": "ERROR", "message": "Database Timeout", "time": "Sat Jun 6 09:52:07 UTC 2026"}
{"server": "AppServer2", "level": "ERROR", "message": "Database Timeout", "time": "Sat Jun 6 09:52:22 UTC 2026"}
{"server": "AppServer2", "level": "ERROR", "message": "Database Timeout", "time": "Sat Jun 6 09:52:37 UTC 2026"}
{"server": "AppServer2", "level": "ERROR", "message": "Database Timeout", "time": "Sat Jun 6 09:52:52 UTC 2026"}
{"server": "AppServer2", "level": "ERROR", "message": "Database Timeout", "time": "Sat Jun 6 09:53:07 UTC 2026"}
{"server": "AppServer2", "level": "ERROR", "message": "Database Timeout", "time": "Sat Jun 6 09:53:22 UTC 2026"}
{"server": "AppServer2", "level": "ERROR", "message": "Database Timeout", "time": "Sat Jun 6 09:53:37 UTC 2026"}
{"server": "AppServer2", "level": "ERROR", "message": "Database Timeout", "time": "Sat Jun 6 09:53:52 UTC 2026"}
root@ip-172-31-11-116 ec2-user:~#
```

i-0c2281169919518f1 (AppServer2)
PublicIPs: 13.207.201.171 PrivateIPs: 172.31.11.116

Step 5: Configure CloudWatch Agent to send logs to CloudWatch Logs.

- Create CloudWatch Agent configuration file.
- Specify log file path.
- Create CloudWatch Log Group:
Centralized Logs.
- Configure log stream names using instance IDs.

CloudWatch > Log management

Log groups

Get-set-go with unified data store

Log groups (1)

By default, we only load up to 10,000 log groups.

Filter log groups or try pattern search

Exact match

Log group	Log class	Anomaly d...	Deletion prot...	Bearer token ...	Data prote...	Sensitive d...	Retention	Metric filters
CentralizedLogs	Standard	Configure	Off	Off	-	-	Never expire	-

Step 6: Verify logs in CloudWatch Log Groups and Streams.

CloudWatch > Log management > CentralizedLogs

Log group details

Log class: Standard

ARN: arn:aws:logs:ap-south-1:973629900070:log-group:CentralizedLogs*

Creation time: 1 day ago

Retention: Never expire

Stored bytes: 471.97 KB

Metric filters: 0

Subscription filters: 0

Contributor Insights rules: -

KMS key ID: -

Deletion protection: Off

Data protection: -

Sensitive data count: -

Custom field indexes: Configure

Transformer: Configure

Anomaly detection: Configure

Bearer token authentication: Off

Log streams

Log streams (2)

By default, we only load the most recent log streams.

Filter log streams or try prefix search

Exact match

Show expired

Log stream	Last event time
i-0c2281169919518f1	2026-06-06 23:06:47 (UTC+05:30)
i-05e02701f7333ad6f	2026-06-06 23:06:34 (UTC+05:30)

The screenshot shows the AWS CloudWatch console interface. The left sidebar contains navigation options like Ingestion, Dashboards, Alarms, AI Operations, GenAI Observability, Application Signals (APM), Infrastructure Monitoring, and Logs. The main panel displays 'Log events' for a specific log group. A search filter is applied, showing a list of events with timestamps and messages. The messages indicate 'Database Timeout' errors from 'AppServer2'.

Timestamp	Message
2026-06-06T23:27:47.848+05:30	{"server":"AppServer2","level":"ERROR","message":"Database Timeout","time":"Sat Jun 6 17:57:42 UTC 2026"}
2026-06-06T23:28:02.848+05:30	{"server":"AppServer2","level":"ERROR","message":"Database Timeout","time":"Sat Jun 6 17:57:57 UTC 2026"}
2026-06-06T23:28:17.848+05:30	{"server":"AppServer2","level":"ERROR","message":"Database Timeout","time":"Sat Jun 6 17:58:12 UTC 2026"}
2026-06-06T23:28:52.848+05:30	{"server":"AppServer2","level":"ERROR","message":"Database Timeout","time":"Sat Jun 6 17:58:28 UTC 2026"}
2026-06-06T23:28:47.848+05:30	{"server":"AppServer2","level":"ERROR","message":"Database Timeout","time":"Sat Jun 6 17:58:43 UTC 2026"}
2026-06-06T23:29:02.848+05:30	{"server":"AppServer2","level":"ERROR","message":"Database Timeout","time":"Sat Jun 6 17:58:58 UTC 2026"}
2026-06-06T23:29:17.848+05:30	{"server":"AppServer2","level":"ERROR","message":"Database Timeout","time":"Sat Jun 6 17:59:13 UTC 2026"}
2026-06-06T23:29:32.848+05:30	{"server":"AppServer2","level":"ERROR","message":"Database Timeout","time":"Sat Jun 6 17:59:28 UTC 2026"}
2026-06-06T23:29:47.848+05:30	{"server":"AppServer2","level":"ERROR","message":"Database Timeout","time":"Sat Jun 6 17:59:43 UTC 2026"}
2026-06-06T23:30:02.848+05:30	{"server":"AppServer2","level":"ERROR","message":"Database Timeout","time":"Sat Jun 6 17:59:58 UTC 2026"}
2026-06-06T23:30:17.848+05:30	{"server":"AppServer2","level":"ERROR","message":"Database Timeout","time":"Sat Jun 6 18:00:13 UTC 2026"}
2026-06-06T23:30:32.848+05:30	{"server":"AppServer2","level":"ERROR","message":"Database Timeout","time":"Sat Jun 6 18:00:28 UTC 2026"}
2026-06-06T23:30:47.848+05:30	{"server":"AppServer2","level":"ERROR","message":"Database Timeout","time":"Sat Jun 6 18:00:43 UTC 2026"}
2026-06-06T23:30:52.848+05:30	{"server":"AppServer2","level":"ERROR","message":"Database Timeout","time":"Sat Jun 6 18:00:58 UTC 2026"}
2026-06-06T23:31:17.848+05:30	{"server":"AppServer2","level":"ERROR","message":"Database Timeout","time":"Sat Jun 6 18:01:13 UTC 2026"}
2026-06-06T23:31:32.848+05:30	{"server":"AppServer2","level":"ERROR","message":"Database Timeout","time":"Sat Jun 6 18:01:28 UTC 2026"}
2026-06-06T23:31:47.848+05:30	{"server":"AppServer2","level":"ERROR","message":"Database Timeout","time":"Sat Jun 6 18:01:43 UTC 2026"}
2026-06-06T23:32:02.848+05:30	{"server":"AppServer2","level":"ERROR","message":"Database Timeout","time":"Sat Jun 6 18:01:58 UTC 2026"}

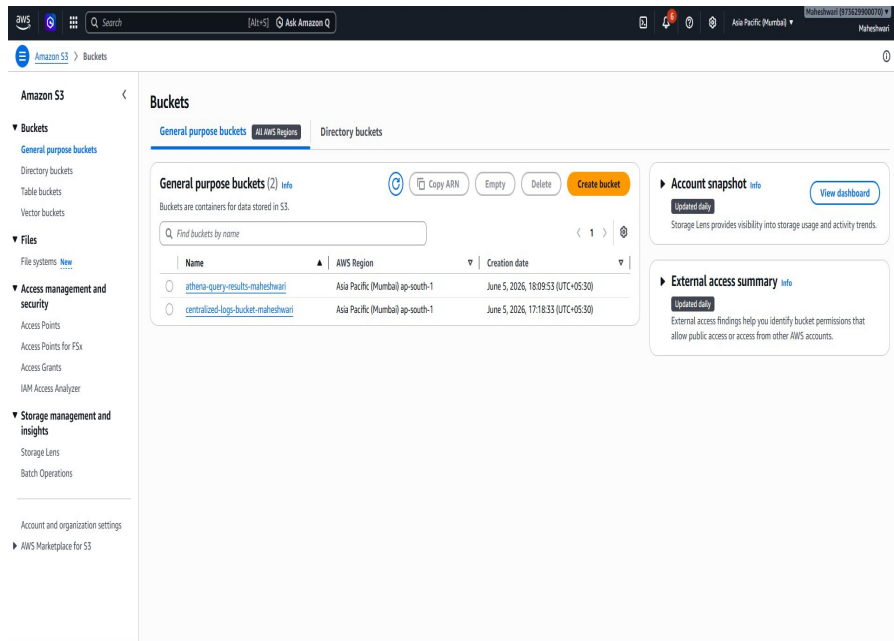
The screenshot shows the AWS CloudWatch console interface. The left sidebar contains navigation options like Ingestion, Dashboards, Alarms, AI Operations, GenAI Observability, Application Signals (APM), Infrastructure Monitoring, and Logs. The main panel displays 'Log events' for a specific log group. A search filter is applied, showing a list of events with timestamps and messages. The messages indicate 'Application Running' status from 'AppServer1'.

Timestamp	Message
2026-06-06T23:24:45.768+05:30	{"server":"AppServer1","level":"INFO","message":"Application Running","time":"Sat Jun 6 17:54:41 UTC 2026"}
2026-06-06T23:24:55.769+05:30	{"server":"AppServer1","level":"INFO","message":"Application Running","time":"Sat Jun 6 17:54:51 UTC 2026"}
2026-06-06T23:25:05.769+05:30	{"server":"AppServer1","level":"INFO","message":"Application Running","time":"Sat Jun 6 17:55:01 UTC 2026"}
2026-06-06T23:25:15.769+05:30	{"server":"AppServer1","level":"INFO","message":"Application Running","time":"Sat Jun 6 17:55:11 UTC 2026"}
2026-06-06T23:25:25.769+05:30	{"server":"AppServer1","level":"INFO","message":"Application Running","time":"Sat Jun 6 17:55:21 UTC 2026"}
2026-06-06T23:25:35.768+05:30	{"server":"AppServer1","level":"INFO","message":"Application Running","time":"Sat Jun 6 17:55:31 UTC 2026"}
2026-06-06T23:25:45.769+05:30	{"server":"AppServer1","level":"INFO","message":"Application Running","time":"Sat Jun 6 17:55:41 UTC 2026"}
2026-06-06T23:25:55.768+05:30	{"server":"AppServer1","level":"INFO","message":"Application Running","time":"Sat Jun 6 17:55:51 UTC 2026"}
2026-06-06T23:26:05.769+05:30	{"server":"AppServer1","level":"INFO","message":"Application Running","time":"Sat Jun 6 17:56:01 UTC 2026"}
2026-06-06T23:26:15.768+05:30	{"server":"AppServer1","level":"INFO","message":"Application Running","time":"Sat Jun 6 17:56:11 UTC 2026"}
2026-06-06T23:26:25.769+05:30	{"server":"AppServer1","level":"INFO","message":"Application Running","time":"Sat Jun 6 17:56:21 UTC 2026"}
2026-06-06T23:26:35.769+05:30	{"server":"AppServer1","level":"INFO","message":"Application Running","time":"Sat Jun 6 17:56:31 UTC 2026"}
2026-06-06T23:26:45.769+05:30	{"server":"AppServer1","level":"INFO","message":"Application Running","time":"Sat Jun 6 17:56:41 UTC 2026"}
2026-06-06T23:26:55.768+05:30	{"server":"AppServer1","level":"INFO","message":"Application Running","time":"Sat Jun 6 17:56:51 UTC 2026"}
2026-06-06T23:27:05.769+05:30	{"server":"AppServer1","level":"INFO","message":"Application Running","time":"Sat Jun 6 17:57:01 UTC 2026"}
2026-06-06T23:27:15.768+05:30	{"server":"AppServer1","level":"INFO","message":"Application Running","time":"Sat Jun 6 17:57:11 UTC 2026"}
2026-06-06T23:27:25.769+05:30	{"server":"AppServer1","level":"INFO","message":"Application Running","time":"Sat Jun 6 17:57:21 UTC 2026"}
2026-06-06T23:27:35.768+05:30	{"server":"AppServer1","level":"INFO","message":"Application Running","time":"Sat Jun 6 17:57:31 UTC 2026"}
2026-06-06T23:27:45.769+05:30	{"server":"AppServer1","level":"INFO","message":"Application Running","time":"Sat Jun 6 17:57:41 UTC 2026"}

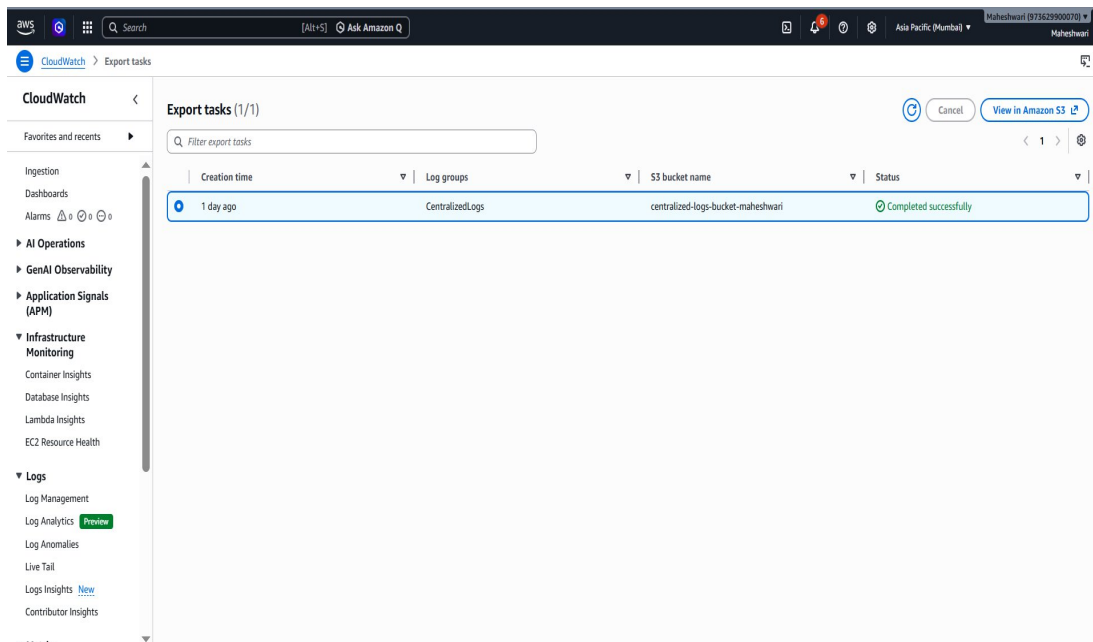
Step 7: Create an S3 bucket for log storage.

- Open S3 Console.
- Create Bucket:
- centralized-logs-bucket.
- Configure bucket settings.

Save bucket.

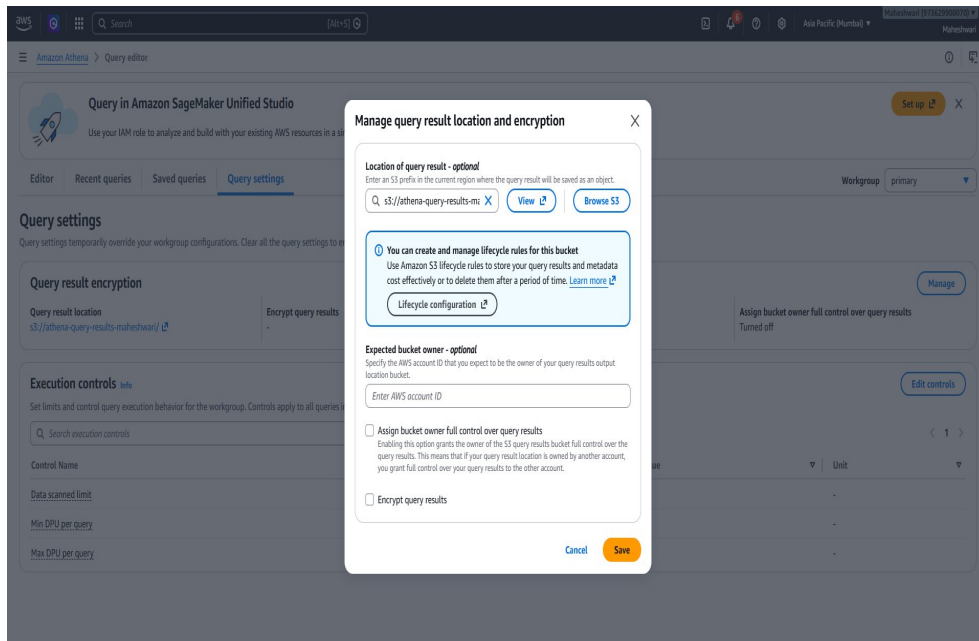


Step 8: Configure log export/storage in S3.



Step 9: Configure Athena query result location and database.

- Open Athena Console.
- Configure query result location.
- Create database:
- centralized logs



Step 10: Create Athena table for log analysis.

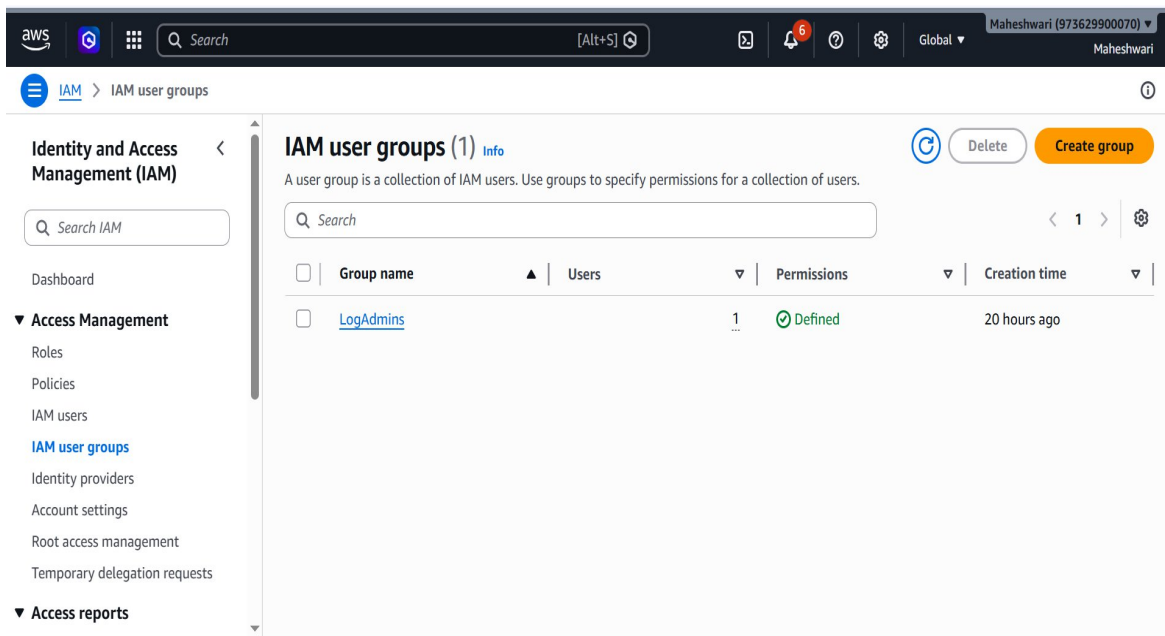
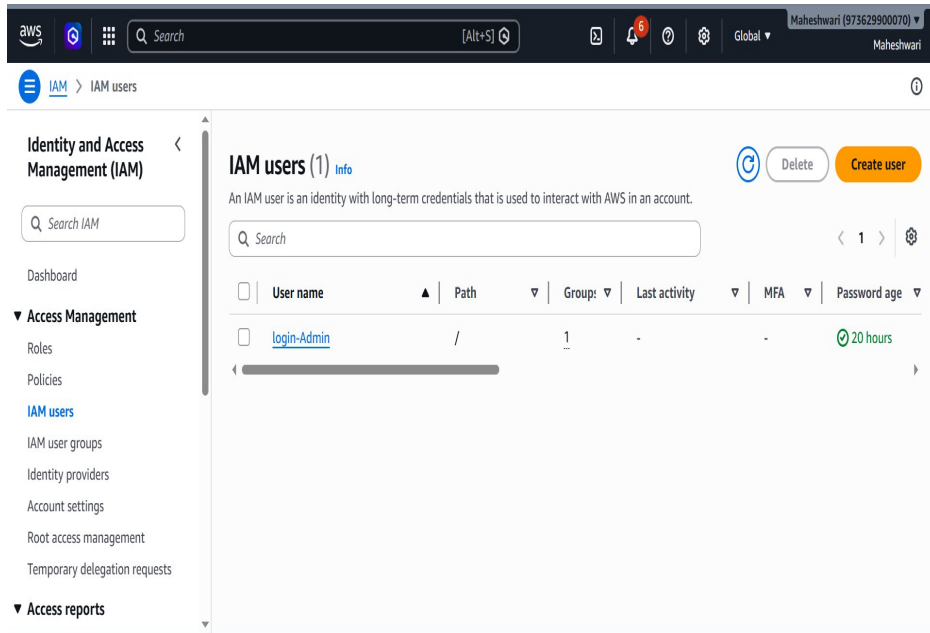
- Create external table pointing to S3 log location.
- Configure schema.
- Save table.

Step 11: Run SQL queries to analyze logs.

- Query log records.
- Count log entries.
- Search error messages.
- Generate operational insights.

Step 12: Configure IAM users, groups and permissions.

- create IAM Group:
- LogAdmins.
- Attach:
- AmazonAthenaFullAccess
- AmazonS3ReadOnlyAccess
- CloudWatchReadOnlyAccess
- Create IAM User.
- Add user to LogAdmins group.



Step 13: Validate Project end-to-end log flow.

- Generate new log entries.
- Verify log flow:
- EC2 → CloudWatch Logs → S3 → Athena.
- Execute Athena queries successfully.

```
root@ip-172-31-8-218 ec2-user# sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl -a status

"status": "Running",
"starttime": "2026-06-06T06:38:42+00:00",
"configstatus": "configured",
"version": "1.300066.2"

root@ip-172-31-8-218 ec2-user# cat /logs/app.log | tail
{"server": "AppServer1", "level": "INFO", "message": "Application Running", "time": "Sat Jun 6 06:40:28 UTC 2026"}
{"server": "AppServer1", "level": "INFO", "message": "Application Running", "time": "Sat Jun 6 06:40:38 UTC 2026"}
{"server": "AppServer1", "level": "INFO", "message": "Application Running", "time": "Sat Jun 6 06:40:48 UTC 2026"}
{"server": "AppServer1", "level": "INFO", "message": "Application Running", "time": "Sat Jun 6 06:40:58 UTC 2026"}
{"server": "AppServer1", "level": "INFO", "message": "Application Running", "time": "Sat Jun 6 06:41:08 UTC 2026"}
{"server": "AppServer1", "level": "INFO", "message": "Application Running", "time": "Sat Jun 6 06:41:18 UTC 2026"}
{"server": "AppServer1", "level": "INFO", "message": "Application Running", "time": "Sat Jun 6 06:41:28 UTC 2026"}
{"server": "AppServer1", "level": "INFO", "message": "Application Running", "time": "Sat Jun 6 06:41:38 UTC 2026"}
{"server": "AppServer1", "level": "INFO", "message": "Application Running", "time": "Sat Jun 6 06:41:48 UTC 2026"}
{"server": "AppServer1", "level": "INFO", "message": "Application Running", "time": "Sat Jun 6 06:41:58 UTC 2026"}
root@ip-172-31-8-218 ec2-user#
```

i-05e02701f7333ad6f (AppServer1)

PublicIPs: 13.206.80.155 PrivateIPs: 172.31.8.218

```
root@ip-172-31-11-116 ec2-user# cat /logs/app.log | tail
{"server": "AppServer2", "level": "ERROR", "message": "Database Timeout", "time": "Sat Jun 6 09:51:37 UTC 2026"}
{"server": "AppServer2", "level": "ERROR", "message": "Database Timeout", "time": "Sat Jun 6 09:51:52 UTC 2026"}
{"server": "AppServer2", "level": "ERROR", "message": "Database Timeout", "time": "Sat Jun 6 09:52:07 UTC 2026"}
{"server": "AppServer2", "level": "ERROR", "message": "Database Timeout", "time": "Sat Jun 6 09:52:22 UTC 2026"}
{"server": "AppServer2", "level": "ERROR", "message": "Database Timeout", "time": "Sat Jun 6 09:52:37 UTC 2026"}
{"server": "AppServer2", "level": "ERROR", "message": "Database Timeout", "time": "Sat Jun 6 09:52:52 UTC 2026"}
{"server": "AppServer2", "level": "ERROR", "message": "Database Timeout", "time": "Sat Jun 6 09:53:07 UTC 2026"}
{"server": "AppServer2", "level": "ERROR", "message": "Database Timeout", "time": "Sat Jun 6 09:53:22 UTC 2026"}
{"server": "AppServer2", "level": "ERROR", "message": "Database Timeout", "time": "Sat Jun 6 09:53:37 UTC 2026"}
{"server": "AppServer2", "level": "ERROR", "message": "Database Timeout", "time": "Sat Jun 6 09:53:52 UTC 2026"}
root@ip-172-31-11-116 ec2-user#
```

i-0c2281169919518f1 (AppServer2)

PublicIPs: 13.207.201.171 PrivateIPs: 172.31.11.116

Amazon S3

Buckets

▼ Buckets

General purpose buckets

Directory buckets

Table buckets

Vector buckets

▼ Files

File systems

▼ Access management and security

Access Points

Access Points for FSx

Access Grants

IAM Access Analyzer

▼ Storage management and insights

Storage Lens

Batch Operations

Account and organization settings

► AWS Marketplace for S3

Buckets

General purpose buckets

ALL AWS Regions

Directory buckets

General purpose buckets (2)

Find buckets by name

Name	AWS Region	Creation date
athena-query-results-maheshwari	Asia Pacific (Mumbai) ap-south-1	June 5, 2026, 18:09:53 (UTC+05:30)
centralized-logs-bucket-maheshwari	Asia Pacific (Mumbai) ap-south-1	June 5, 2026, 17:18:33 (UTC+05:30)

Account snapshot

External access summary

Amazon S3

Buckets

General purpose buckets

Directory buckets

Table buckets

Vector buckets

Files

File systems

Access management and security

Access Points

Access Points for FSx

Access Grants

IAM Access Analyzer

Storage management and insights

Storage Lens

Batch Operations

Account and organization settings

AWS Marketplace for S3

05/

Objects

Properties

Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

Objects (10)

Find objects by prefix

1

☐	Name	Type	Last modified	Size	Storage class
☐	10575504-04f3-4d7d-82af-2a7ad65738cc.txt	txt	June 5, 2026, 18:19:04 (UTC+05:30)	0 B	Standard
☐	54a66a99-b00d-4222-9faa-e208839672c6.csv	csv	June 5, 2026, 18:21:54 (UTC+05:30)	10.0 B	Standard
☐	54a66a99-b00d-4222-9faa-e208839672c6.csv.metadata	metadata	June 5, 2026, 18:21:54 (UTC+05:30)	76.0 B	Standard
☐	5ec89f93-29aa-43a6-bba2-208ae24ae08.csv	csv	June 5, 2026, 18:33:20 (UTC+05:30)	40.0 B	Standard
☐	5ec89f93-29aa-43a6-bba2-208ae24ae08.csv.metadata	metadata	June 5, 2026, 18:33:20 (UTC+05:30)	76.0 B	Standard
☐	842d11cb-8f08-40ac-9514-4880094a3d74.csv	csv	June 5, 2026, 18:20:42 (UTC+05:30)	40.0 B	Standard
☐	842d11cb-8f08-40ac-9514-4880094a3d74.csv.metadata	metadata	June 5, 2026, 18:20:42 (UTC+05:30)	76.0 B	Standard
☐	91cd0df5-bc6f-4ac4-87ec-07f8b6d40a06.csv	csv	June 5, 2026, 18:22:17 (UTC+05:30)	12.0 B	Standard
☐	91cd0df5-bc6f-4ac4-87ec-07f8b6d40a06.csv.metadata	metadata	June 5, 2026, 18:22:17 (UTC+05:30)	67.0 B	Standard

CloudWatch

Log group details

Log class

Standard

ARN

arn:aws:logs:ap-south-1:97362990070:log-group:CentralizedLogs*

Creation time

21 hours ago

Retention

Never expire

Stored bytes

471.57 KB

Metric filters

0

Subscription filters

0

Contributor Insights rules

-

KMS key ID

-

Deletion protection

Off

Data protection

-

Sensitive data count

-

Custom field indexes

Configure

Transformer

Configure

Anomaly detection

Configure

Bearer token authentication

Off

Log streams

Tags

Data protection

Anomaly detection

Metric filters

Subscription filters

Contributor Insights

Field indexes

Transformer

Log streams (2)

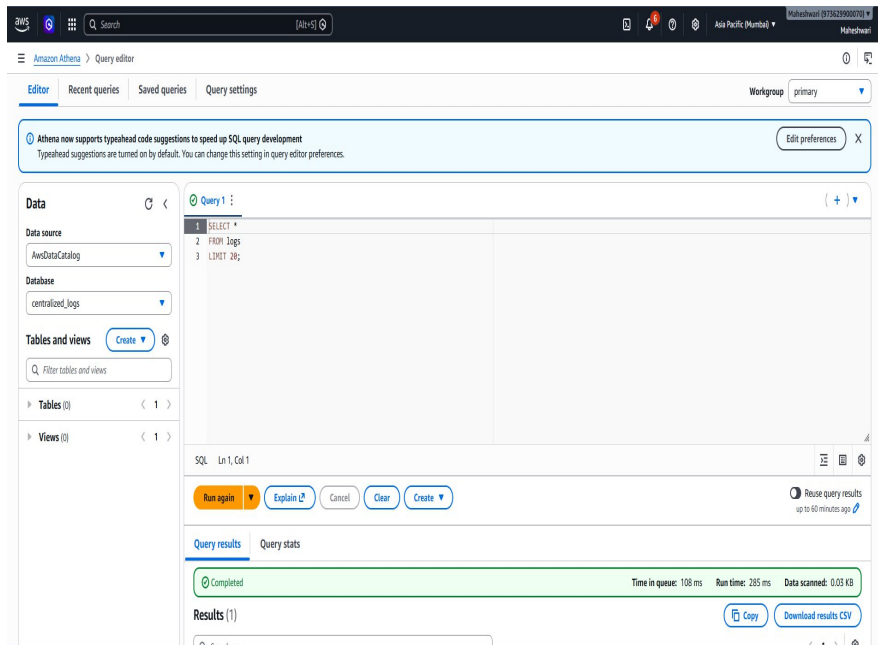
Filter log streams or try prefix search

Exact match

Show expired

1

☐	Log stream	Last event time
☐	i-05e0270177333ad6f	2026-06-06 14:56:26 (UTC+05:30)
☐	i-0c22811699195181	2026-06-06 14:55:26 (UTC+05:30)



Troubleshooting

Issue 1: CloudWatch Agent Not Sending Logs

Cause: Incorrect CloudWatch Agent configuration.

Solution:

- Verify log file path.
- Restart CloudWatch Agent.
- Check CloudWatch Agent logs.
- Confirm IAM role permissions.

Issue 2: Log Group Not Receiving Logs

Cause: CloudWatch Agent service not running.

Solution:

- Verify agent status.
- Restart service.
- Confirm network connectivity.

Issue 3: Athena Returns No Data

Cause: Incorrect S3 path or table schema.

Solution:

- Verify S3 bucket path.
- Check Athena table definition.
- Confirm log files exist in S3.

Issue 4: Access Denied Errors

Cause: Insufficient IAM permissions

Solution: Review IAM policies.

Attach required CloudWatch, S3, and Athena permissions.

Conclusion

The Centralized Logging and Analytics System provides a scalable and efficient solution for collecting, storing, and analyzing logs from multiple EC2 instances. By integrating CloudWatch Logs, S3, Athena, and IAM, the system enables centralized monitoring, faster troubleshooting, and secure log management. The solution is cost-effective, highly scalable, and suitable for real-world DevOps and Cloud Operations environments while remaining appropriate for AWS Free Tier learning projects.